

Molecular Dynamics study of FCC crystal

Sokratis Pastelas

MSc student in Computational Physics, Auth

Table of Contents

Overview	3
Theoretical Background.....	4
Molecular Dynamics.....	4
Lennard - Jones Potential	4
Velocity Verlet algorithm	5
Berendsen thermostat	6
Lennard - Jones units	6
Physics constants calculation	7
Computational complexity of force calculations	8
Neighbor list algorithm	8
OpenGL and supporting libraries	10
Graphics pipeline	10
Implementation details	12
System visualization	12
Program pipeline	12
Results.....	13
Impact of τ on system behavior	13
System response to different target temperatures.....	14
Simulation results for different numbers of particles	15
Impact of r_{skin} on simulation time.....	17
Energy evolution during the simulation.....	18
Temperature, Energy and Pressure stability	19
Conclusion.....	21
Future improvements	22
References.....	23
Image References	23

Overview

This project presents the development of a molecular dynamics (MD) simulation to investigate the melting behavior of a face-centered cubic (FCC) crystal. The simulation was implemented in C++ and combines computational physics techniques for calculating atomic trajectories with real-time visualization using OpenGL.

Interatomic interactions are described using the Lennard–Jones (LJ) potential. The equations of motion are integrated using the Velocity Verlet algorithm, providing accurate and numerically stable particle trajectories. Moreover, temperature control is achieved through the Berendsen thermostat, allowing the system to maintain a desired target temperature.

To improve computational efficiency, the simulation employs the neighbor list algorithm. By avoiding unnecessary pairwise interaction calculations, the algorithm significantly reduces the computational cost. As the number of atoms in the system increases, the performance benefits become even more pronounced, allowing the simulation to scale efficiently and handle larger systems than a brute-force approach.

The simulation initializes the atoms in a face-centered cubic lattice (FCC), a common crystal structure found in many metals. As the temperature increases, the atomic arrangement gradually loses its ordered structure, allowing the melting process and phase transition to be observed and analyzed. Throughout the simulation, important physical quantities such as kinetic energy, potential energy, temperature, and pressure are calculated to monitor the thermodynamic state of the system.

For visualization, the project uses OpenGL with GLFW, GLEW, and GLM to render the atoms in a three-dimensional environment. Real-time rendering allows the evolution of the crystal structure to be observed interactively while the simulation is running.

Overall, the project demonstrates the integration of molecular dynamics algorithms, efficient numerical techniques, and modern graphics programming to create an interactive simulation capable of modeling the thermal behavior and melting of an FCC crystal.

Theoretical Background

Molecular Dynamics

Molecular Dynamics (MD) is a computational simulation method used to study the motion and interactions of atoms and molecules over time. It is based on Newton's laws of motion, where the forces acting on each particle are calculated from an interatomic potential, and the resulting equations of motion are numerically integrated to determine their trajectories. By repeatedly updating particle positions and velocities, molecular dynamics enables the investigation of the structural, thermodynamic, and dynamic properties of materials at the atomic scale. It is widely used in materials science, chemistry and physics to study phenomena such as diffusion, phase transitions, crystal deformation, and melting.

Lennard - Jones Potential

The Lennard–Jones (LJ) potential is a widely used mathematical model for describing the interaction between neutral atoms and molecules. Out of all the intermolecular potentials, it is probably the one that has been the most extensively studied. The potential depends only on the distance between two particles and is expressed by the equation:

$$V_{ij}(r) = 4\varepsilon \left[\frac{\sigma^{12}}{r^{12}} - \frac{\sigma^6}{r^6} \right]$$

, where (r) is the distance between two atoms, ε defines the depth of the potential well, and σ represents the characteristic distance at which the potential is zero.

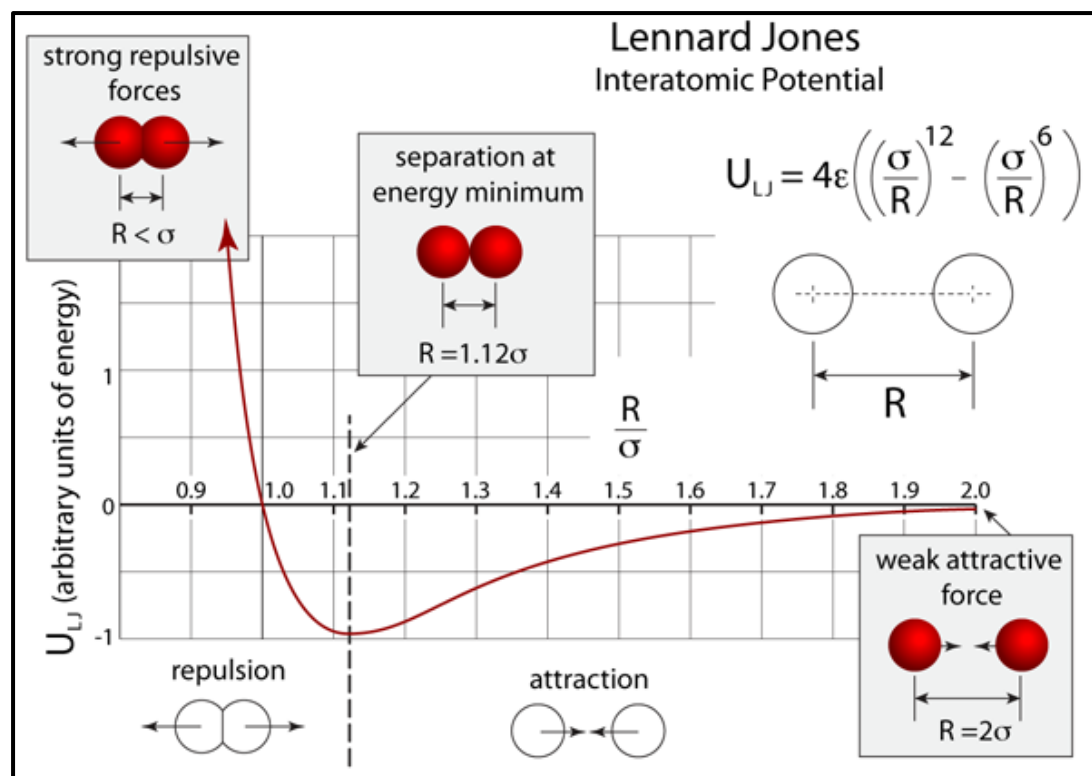


Figure 1: Lennard - Jones potential¹.

In this project, the Lennard–Jones potential is used to calculate the forces acting between neighboring atoms, which determine their motion throughout the simulation. Although it is a simplified model compared to more advanced interatomic potentials.

Velocity Verlet algorithm

In this project the Velocity Verlet algorithm is used in order to solve numerically the Newton's equations of motion. The algorithm is a numerical integration method used to update the positions and velocities of particles in molecular dynamics simulations. It is based on Newton's second law and provides second-order accuracy in time while maintaining good numerical stability and energy conservation over long simulations. In each timestep, particle positions are first updated using their current velocities and accelerations. New forces are then calculated from the updated positions, and the velocities are subsequently corrected using the average of the old and new accelerations. This approach allows the algorithm to accurately account for the changing forces acting on the particles while remaining computationally efficient. Due to its stability, simplicity, and favorable energy conservation properties, the Velocity Verlet algorithm is one of the most widely used

integration schemes in molecular dynamics. The algorithm consists of the following steps:

1. Update particle positions: $r(t + \Delta t) = r(t) + v(t)\Delta t + \frac{1}{2}a(t)\Delta t^2$
2. Compute new forces and acceleration: $a(t + \Delta t) = \frac{F(t+\Delta t)}{m}$
3. Update particle velocities: $v(t + \Delta t) = v(t) + \frac{a(t)+a(t+\Delta t)}{2}\Delta t$

Berendsen thermostat

The Berendsen thermostat is a temperature control algorithm commonly used in molecular dynamics simulations to regulate the system temperature. It operates by continuously rescaling particle velocities by a factor λ , so that the instantaneous temperature gradually approaches a specified target value. The rate at which the temperature is adjusted is controlled by a relaxation time parameter (τ), allowing the system to be heated or cooled smoothly without introducing abrupt changes in particle motion. The scaling factor λ is given by the equation:

$$\lambda = \sqrt{1 + \frac{\Delta t}{\tau} \left(\frac{T_0}{T} - 1 \right)}$$

, where T_0 is the target temperature, T the current temperature and Δt the time-step of the simulation.

Due to its simplicity and computational efficiency, it was used in this project to control the temperature of the FCC crystal.

Lennard - Jones units

In molecular dynamics, Lennard-Jones (LJ) units constitute a system of reduced, dimensionless units defined by selecting characteristic parameters of the Lennard-Jones potential as fundamental scales. Specifically, the length scale is set by $\sigma = 1$, corresponding to the distance at which the interparticle potential becomes zero, the energy scale by $\epsilon = 1$, representing the depth of the potential well, and the mass scale by $m = 1$. Within this framework, distances are measured in units of σ , energies in units of ϵ , and temperature in units of $\frac{\epsilon}{k_B}$,

while the natural time unit is given by $\tau = \sigma \sqrt{\frac{m}{\epsilon}}$.

This nondimensionalization not only simplifies computational implementation but also introduces a degree of universality, allowing simulation results to be transferred between different physical systems through appropriate rescaling of LJ parameters.

Physics constants calculation

In molecular dynamics simulations, the potential energy is computed as the sum of all pairwise interaction energies between particles, where each interacting pair contributes an energy that depends on their separation distance. So, it follows the formula:

$$E_{pot} = \sum_{i < j} V(r_{ij})$$

The kinetic energy is obtained from the instantaneous particle velocities through the classical expression:

$$E_{kin} = \sum_{i=1}^N \frac{1}{2} m u_i^2$$

, where N is the number of atoms with mass m and u_i the magnitude of the velocity of atom-i. The temperature is then defined as:

$$T = \frac{2}{3Nk_B} \sum_{i=1}^N \frac{1}{2} m u_i^2$$

Finally, the pressure can be calculated using the virial theorem, combining both kinetic and configurational contributions. It depends on particle momenta as well as interparticle forces and distances, and is expressed as:

$$P = \frac{Nk_B T}{V} + \frac{W}{V}$$

, where V is the volume of the system and:

$$W = \sum_{i < j} r_{ij} \cdot F_{ij}$$

In the pressure formula the first term corresponds to the ideal gas contribution and the second accounts for intermolecular interactions.

Computational complexity of force calculations

One of the main computational challenges in molecular dynamics simulations is the calculation of interatomic forces. In a system containing N atoms, a straightforward approach requires each atom to interact with every other atom. Even though all interactions between atoms with distance larger than some predefined distance r_{cutoff} are considered negligible, all the pairs must be checked. This results in $\frac{N \cdot (N-1)}{2}$ pairwise force calculations at every simulation step. Consequently, the computational complexity grows as $O(N^2)$, meaning that the execution time increases rapidly as the number of atoms becomes larger. This quadratic scaling makes force calculations impractical for large systems and motivates the use of more efficient algorithms to reduce the number of interactions that must be evaluated.

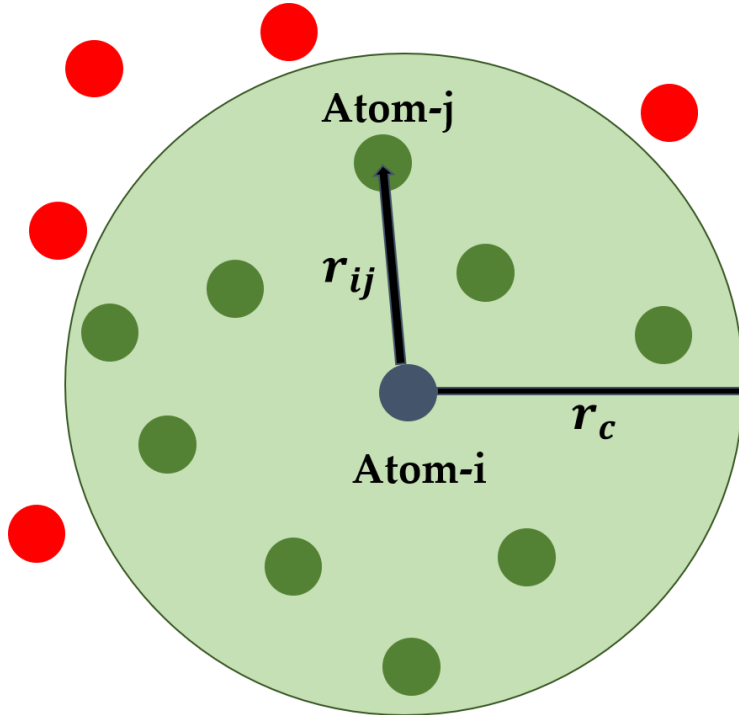


Figure 2: Atoms outside the cutoff radius (red) are considered negligible for the force calculations.

Neighbor list algorithm

To overcome the $O(N^2)$ scaling of direct force calculations, the simulation employs the neighbor list algorithm. The main idea is to restrict force evaluations only to atoms that are within a specified list radius (r_{list}) distance. The r_{list} is equal to the r_{cutoff} plus a small increment r_{skin} so for each atom the unnecessary checks are significantly reduced. For each atom, a list of nearby

neighbors is constructed and updated when needed during the simulation. On force computation, interactions are then evaluated only between atoms in each particle's neighbor list, significantly reducing the number of pairwise calculations. This approach improves computational efficiency and allows the simulation to scale as $\sim O(N)$, enabling the program to simulate larger systems.

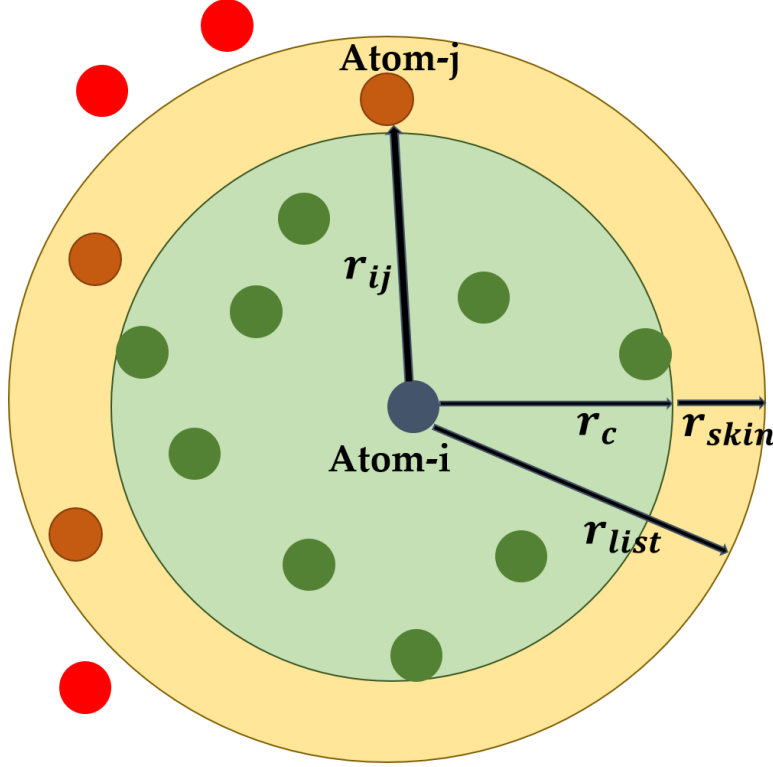


Figure 3: Neighbor list algorithm. The atom-j is included in the list of neighbor atoms of atom-i. The atoms shown in red are not included.

The neighbor list is not updated at every simulation step, as this would minimize its computational advantage. Instead, it is rebuilt when some particle has moved beyond a predefined safety margin which equals $\frac{r_{skin}}{2}$. This margin ensures that no relevant interactions are missed between updates as atoms move within the system. In practice, the value of r_{skin} is chosen as a trade-off between computational efficiency and accuracy. Choosing a small r_{skin} value causes the neighbor lists to update too often significantly increasing the computational cost, on the other hand a large r_{skin} leads to many unnecessary checks which can also increase the computational time needed.

OpenGL and supporting libraries

OpenGL is a cross-platform graphics API used for rendering 2D and 3D graphics by interacting directly with the GPU. It provides a low-level interface that allows developers to define geometric data and shaders to communicate with the GPU and display complex visual scenes. Nowadays it is maintained by the Khronos Group, an industry group whose members include major technology companies such as NVIDIA, AMD, Intel, and Qualcomm. In this project, OpenGL is used to visualize the molecular dynamics simulation in real time.

Several supporting libraries are used alongside OpenGL to simplify development. GLFW is responsible for creating windows, handling user input, and managing the OpenGL context. GLEW is used to load modern OpenGL functions that are not directly accessible through the default system interface. GLM is a mathematics library designed for graphics applications, providing vector and matrix operations required for transformations and camera control. Finally, ImGui is used to create an interactive graphical user interface, allowing the real-time monitoring of simulation parameters such as temperature and visualization settings.

Graphics pipeline

The OpenGL graphics pipeline describes the sequence of stages used to transform 3D data into a final 2D image displayed on the screen. First, vertex data such as positions and attributes are sent from the application to the GPU. In the vertex shader stage, each vertex is transformed using various linear algebra calculations. After this, the primitive assembly and rasterization stages convert geometric primitives into fragments that correspond to screen pixels.

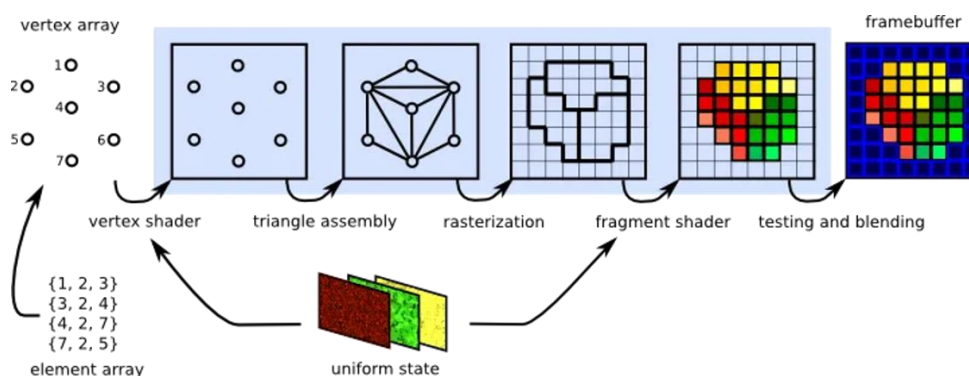


Figure 4: OpenGL graphics pipeline².

Each fragment is then processed by the fragment shader, which determines its final color based on lighting, textures, and other visual properties. Then, many tests such as the depth test are applied to ensure correct visibility of

overlapping objects, finally the resulting pixel values are written to the framebuffer for display. This pipeline ensures efficient parallel processing on the GPU and forms the foundation of modern real-time rendering systems.

Implementation details

System visualization

The molecular dynamics simulation was implemented in C++ and structured around a main loop that updates the physical state of the system and renders it at each frame. The visualization is performed using OpenGL, where each particle is rendered as a sphere to represent an atom in the system. The application also includes several interactive controls, such as an orbital camera that can be rotated using the right mouse button and zoomed in or out using the scroll wheel. In addition, each atom is color-coded based on its velocity, providing a visual representation of particle dynamics. Finally some useful variables are displayed on the Dear ImGui window.

Program pipeline

The program follows a structured pipeline that combines initialization, computation, and rendering in a continuous loop. Initially, particle data are loaded into the GPU, and some necessary simulation parameters are defined such as the atomic positions in the FCC lattice and the random initial velocities of the atoms. Once initialization is complete, the program enters the main simulation loop.

During each iteration, the system first performs the Velocity Verlet integration step to update particle positions and velocities based on interatomic forces. Physical quantities such as kinetic energy, potential energy, temperature, and pressure are then computed and written to an output file for. Finally, the updated particle positions and properties, such as velocity-based color, are sent to the GPU for rendering using OpenGL, allowing real-time visualization of the system's evolution.

This loop continues until the maximum number of simulation steps is reached or the simulation is manually stopped by the user.

Results

Impact of τ on system behavior

The system's behavior was analyzed with respect to variations in the parameter τ , which controls the relaxation rate of the velocity updates and therefore influences how quickly the system responds to changes in temperature. A larger value of τ results in slower adjustments of particle velocities and therefore in system's temperature, while a smaller value leads to more rapid changes. The simulation was run for 3000 steps.

Three different values were tested, $\tau = 0.05$, $\tau = 0.1$, and $\tau = 0.5$, with a target temperature of 1 (Lennard–Jones units). It was observed that for $\tau = 0.5$, the system evolves too slowly to reach the target temperature within the available simulation time. In contrast, for $\tau = 0.1$ and $\tau = 0.05$, the system successfully reaches the desired temperature, with the smaller value achieving equilibration more rapidly. In both successful cases, the temperature stabilizes around the target value of 1. For the later experiments the value of $\tau = 0.1$ was used.

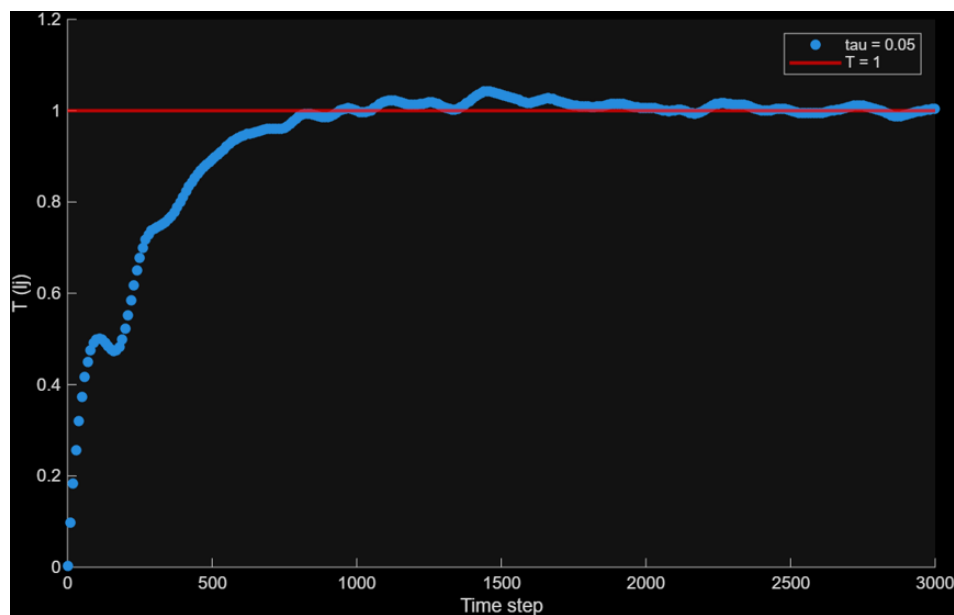
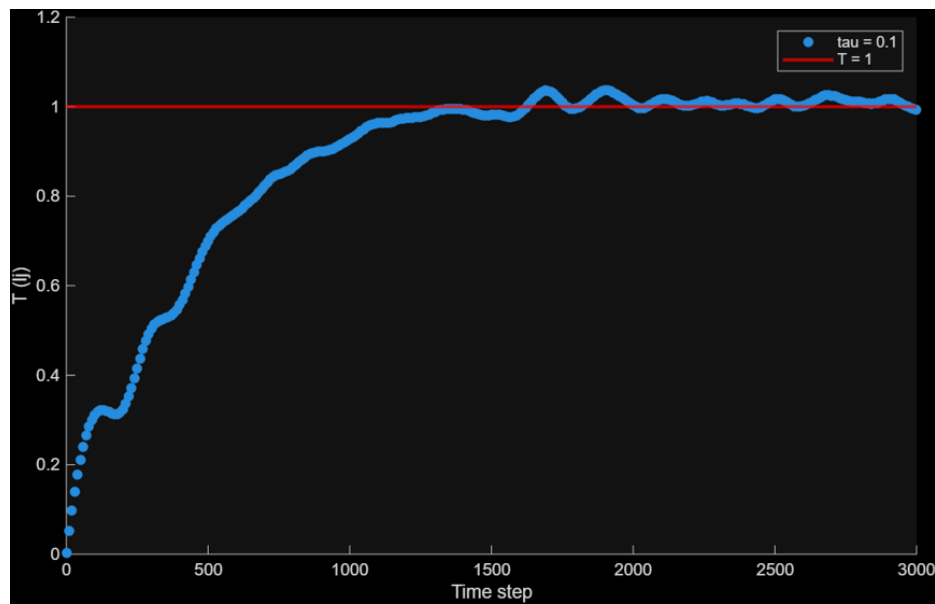
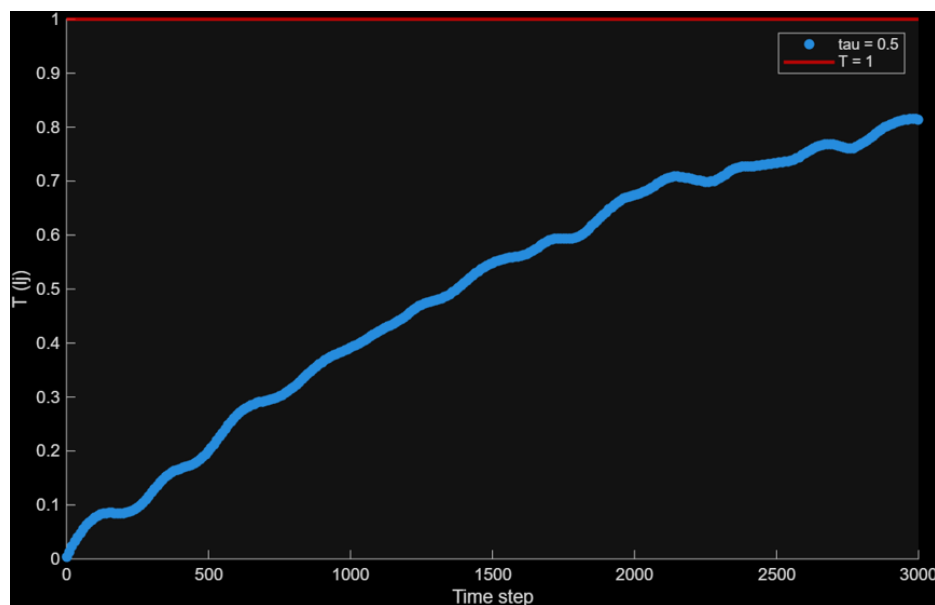


Figure 5: $\tau = 0.05$.

Figure 6: $\tau = 0.1$.Figure 7: $\tau = 0.5$.

System response to different target temperatures

We investigated the effect of different target temperatures $T = 1$, $T = 3$, and $T = 5$ (lj units). In all cases, the system successfully reached and maintained the desired temperature. It was also observed that lower target temperatures were reached more quickly, while higher temperatures required a longer equilibration time.

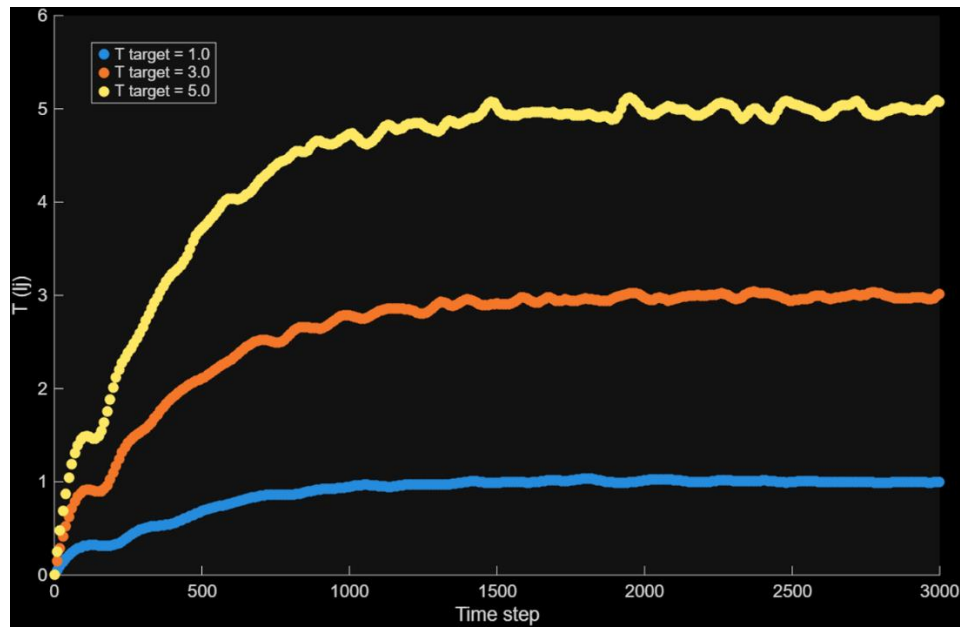


Figure 8: System response to different target temperatures.

In the subsequent experiments, a target temperature of $T = 1$ will be used, as it is sufficiently high to induce melting in the system while still allowing stable equilibration and observation of the phase transition.

Simulation results for different numbers of particles

The simulation was performed for different system sizes, with the number of particles set to $N = 256, 500, 864, 1372, 2048, 2916$. These values were chosen to construct complete face-centered cubic (FCC) crystals without vacancies. In an FCC lattice, each unit cell contains four atoms, and therefore the total number of atoms in a fully filled cubic system follows the relation:

$$N = 4n^3, n = 1, 2, \dots$$

, where n represents the number of unit cells along each spatial dimension.

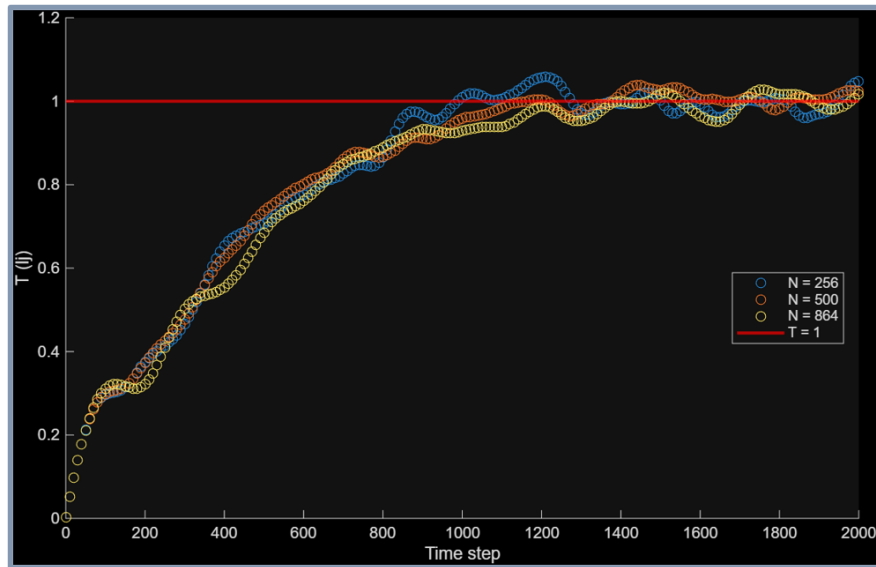


Figure 9: Temperature evolution for different N .

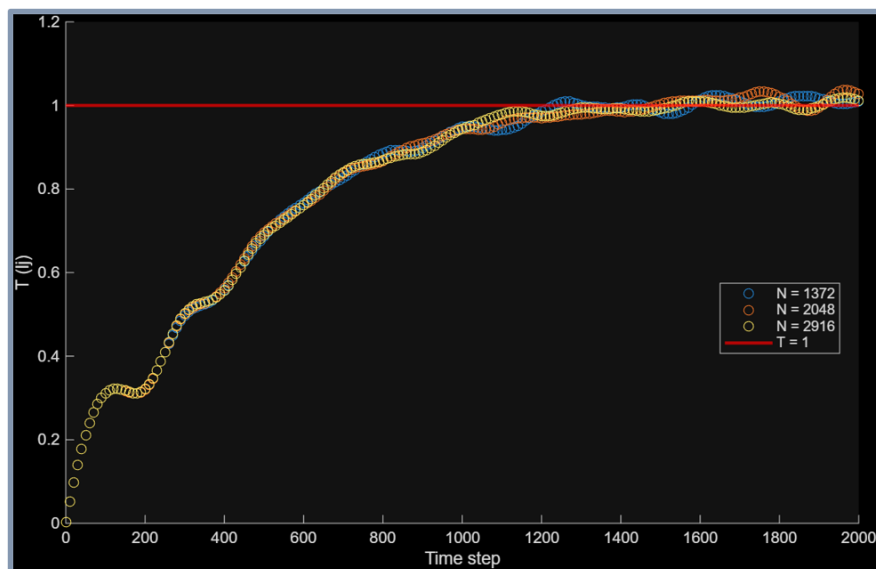


Figure 10: Temperature evolution for different N .

For each system size N , the target temperature was reached in approximately the same amount of simulation time. This indicates that the equilibration rate is largely independent of the number of particles within the tested range. Although larger systems contain more degrees of freedom, the thermostat acts globally on the system, ensuring consistent temperature relaxation behavior across different values of N .

Additionally, we plotted the average time per frame as a function of the number of particles N . The results show that the computational time increases approximately linearly with system size, as expected when using the neighbor list algorithm. This linear scaling indicates that the optimization is effective, since it avoids the quadratic $O(N^2)$ cost of direct pairwise force calculations. So, it successfully improves the scalability of the simulation for larger systems.

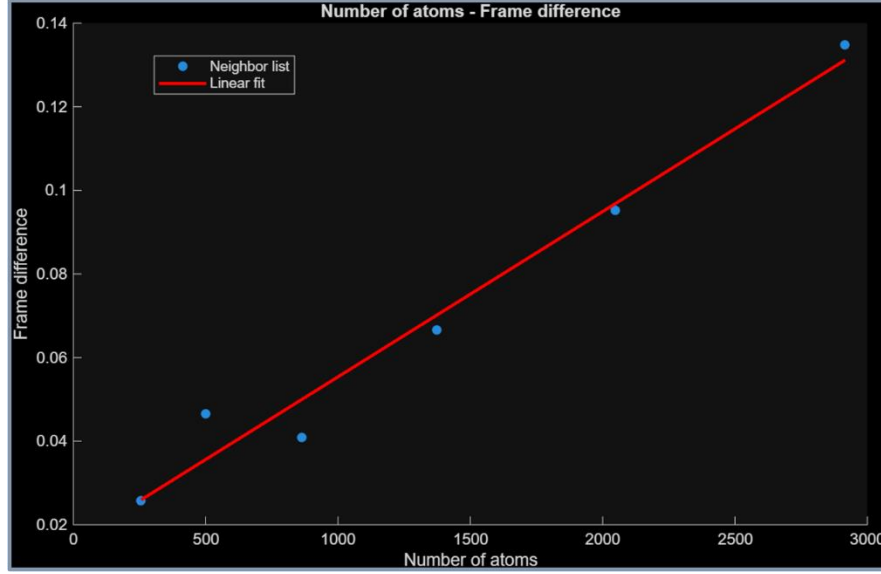


Figure 11: Average time per frame vs N . The data show a linear relationship.

Impact of r_{skin} on simulation time

As mentioned previously, the magnitude of r_{skin} has a significant impact on computational performance. To analyze this effect, we measured the average time per frame for different values of r_{skin} . The tested values were 0.01, 0.1, 0.3, and 0.5. For $r_{skin} = 0.01$, the neighbor lists are rebuilt too frequently, leading to increased computational overhead. In contrast, for $r_{skin} = 0.5$, the neighbor lists become too large, resulting in many unnecessary pair checks and again increasing the computational cost. The best performance was observed for intermediate values, particularly in the range between 0.1 and 0.3, where a good balance between update frequency and lists' size is achieved.

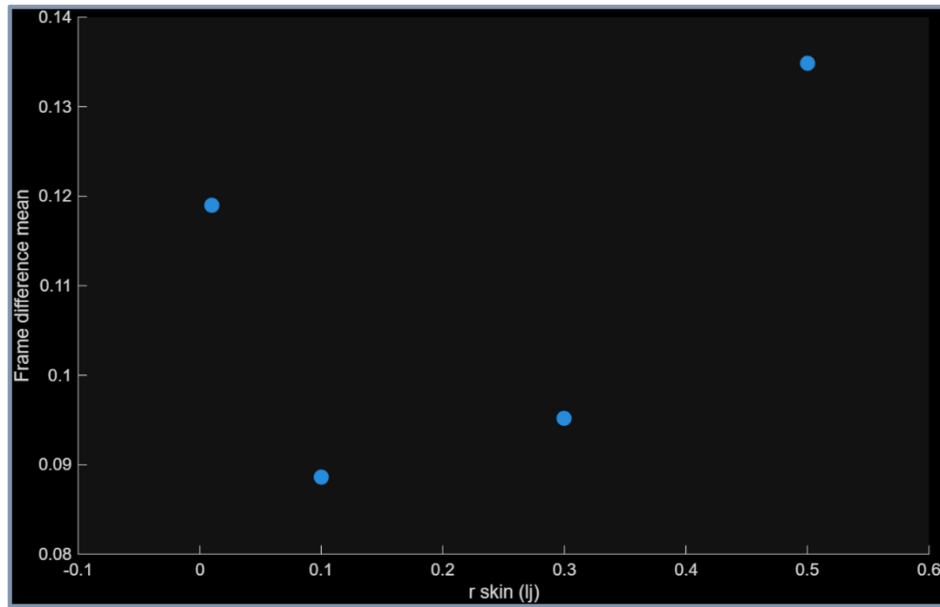


Figure 12: Average time per frame vs r_{skin} .

Energy evolution during the simulation

The kinetic, potential, and total energies of the system were monitored over 2000 simulation timestep. The results were obtained using a system of $N = 2048$ particles, a target temperature of $T = 1$ and a thermostat relaxation parameter of $\tau = 0.1$. During the initial equilibration period, the kinetic and potential energies adjust as the thermostat drives the system toward the target temperature. After equilibration, all three energy components fluctuate around stable mean values, indicating that the system has reached thermal equilibrium.

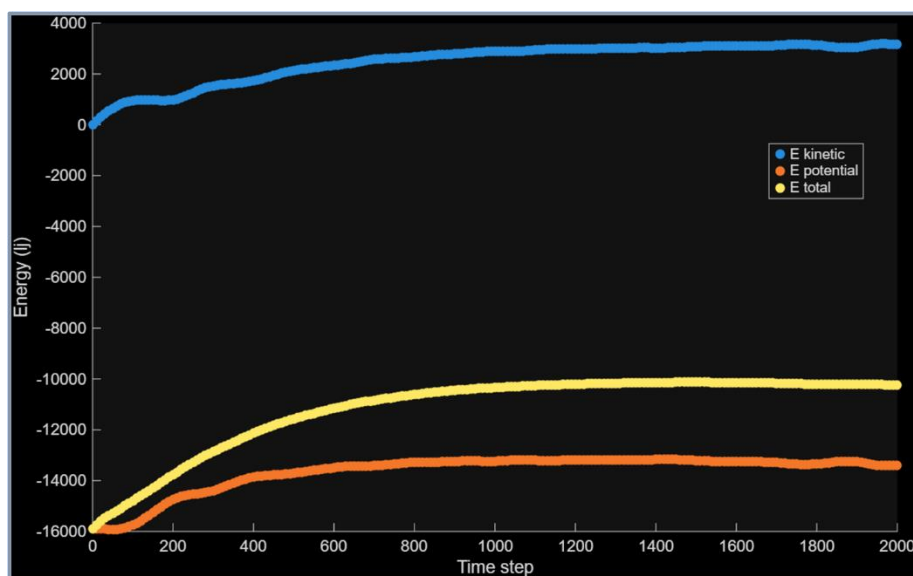


Figure 13: Time step vs Kinetic, Potential, and Total energy.

Temperature, Energy and Pressure stability

The temperature, pressure, and total energy of the system were monitored over 6000 simulation timesteps using the same configuration as in the previous experiment. The temperature and total energy exhibit the expected behavior, reaching equilibrium after an initial transient period and then fluctuating around stable mean values. In contrast, the pressure displays significantly larger fluctuations throughout the simulation.

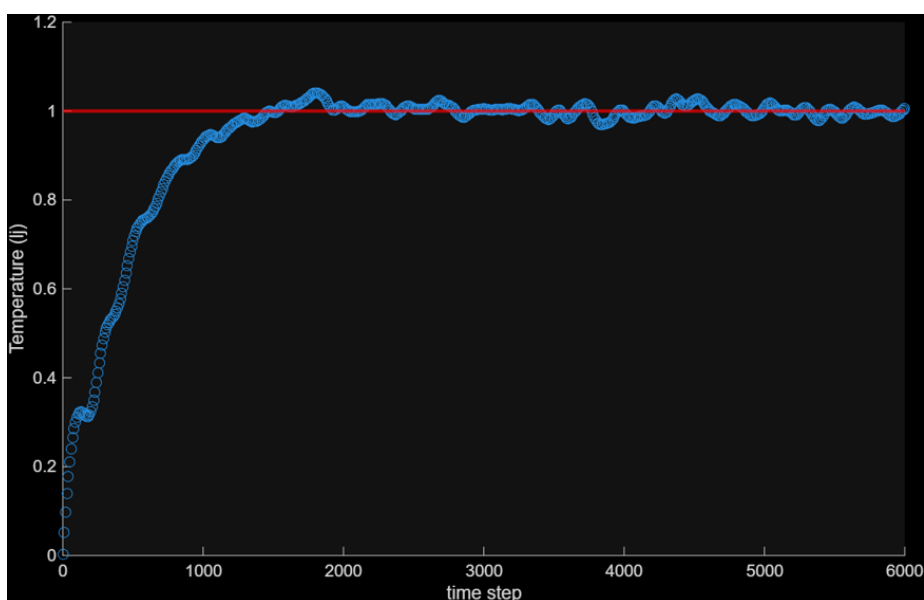


Figure 14: Time step vs Temperature.

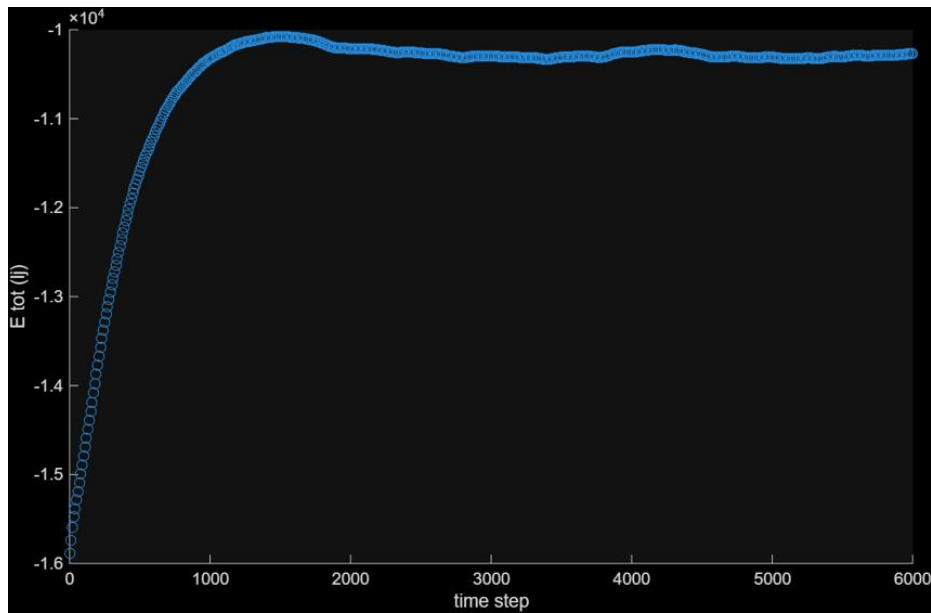


Figure 15: Time step vs Total Energy.

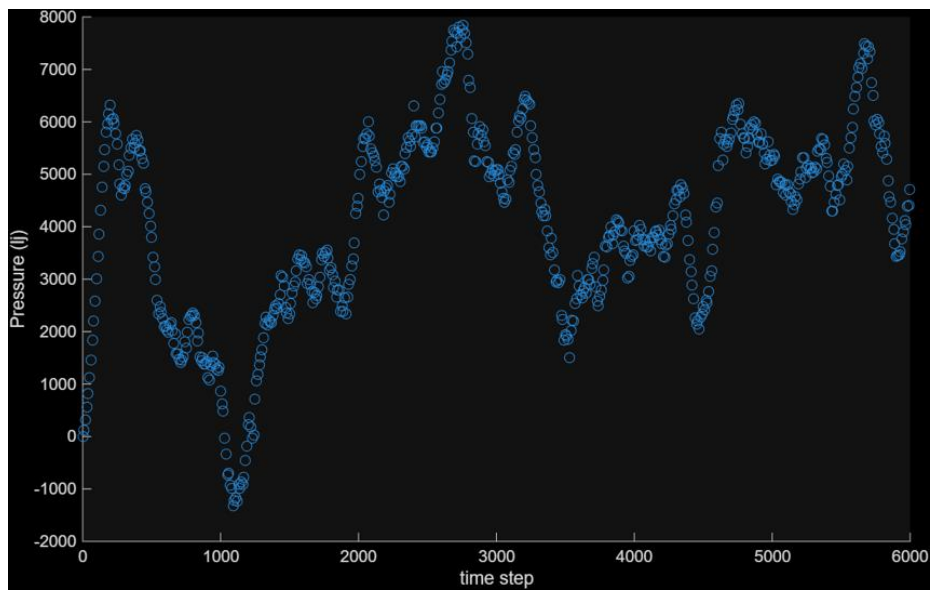


Figure 16: Time step vs Pressure.

These fluctuations are mainly attributed to the virial term W , in the pressure equation. Since W depends on the instantaneous interatomic distances and interaction forces, it is highly sensitive to even small atomic rearrangements. As atoms continuously vibrate and move within the system, the virial contribution can vary substantially, leading to pressure fluctuations. Although some degree of pressure fluctuation is expected in molecular dynamics simulations, the magnitude observed in this case is relatively large. This problem could be attributed to the Berendsen thermostat, which does not rigorously reproduce the fluctuations of the canonical ensemble.

Conclusion

This project successfully achieved its primary objective of developing a molecular dynamics simulation capable of reproducing the melting behavior of a FCC crystal. The simulator was implemented in C++ and combines physically based molecular dynamics calculations with real-time visualization using OpenGL.

The use of the Lennard–Jones potential, the Velocity Verlet integration scheme, and the Berendsen thermostat enabled stable and efficient simulation of particle dynamics. The system successfully reached and maintained the desired target temperatures, while the evolution of kinetic, potential, and total energy demonstrated stable behavior after the equilibration period. Although the Berendsen thermostat effectively controlled the system temperature, the calculated pressure exhibited significant fluctuations. This behavior is primarily attributed to the virial term in the pressure equation, which is highly sensitive to small variations in interatomic distances and forces.

A major contribution of this project is the implementation of the neighbor list algorithm, which substantially improved computational performance. Performance measurements showed that the computational time increased approximately linearly with the number of particles, demonstrating the expected $O(N)$ scaling rather than the $O(N^2)$ behavior of a brute-force implementation. This optimization makes the simulator suitable for significantly larger systems.

The visualization part, implemented with OpenGL, enabled smooth real-time rendering of the atomic system. Since OpenGL is a low-level graphics API that communicates efficiently with the GPU, it provides high rendering performance. This made it possible to observe the melting process and the continuous motion of atoms in real time throughout the simulation.

Concluding, the project demonstrates that combining efficient molecular dynamics algorithms with optimized real-time graphics provides a powerful tool for studying materials at atomic scale. The implemented simulation forms a solid foundation for future extensions involving more advanced physical models, larger systems, and additional performance optimizations.

Future improvements

Several improvements can be made to further enhance both the performance and the capabilities of the simulator. A natural extension is the parallelization of the molecular dynamics calculations using libraries such as MPI and OpenMP. This would provide a significant performance improvement on multi-core computing systems. The graphical user interface could also be expanded to include additional simulation controls and real-time visualization of more thermodynamic quantities. From a physical modeling perspective, replacing the Berendsen thermostat with a thermostat that more accurately reproduces the canonical ensemble would improve the simulations. Furthermore, the implementation of a barostat would enable constant pressure (NPT) simulations. Finally, support for crystal structures beyond FCC could be added by implementing a POSCAR file parser, enabling the simulation of a much wider range of materials and lattice geometries.

References

1. D. C. Rapaport, *The Art of Molecular Dynamics Simulation*. Cambridge, UK: Cambridge University Press, 2nd edition, 2004.
2. Callister, W. D., and Rethwisch, D. G. *Materials Science and Engineering: An Introduction*. Wiley, 10th edition.
3. <https://learnopengl.com/>
4. <https://docs.lammps.org/Manual.html/>

Image References

1. <https://physicsatmcl.commons.msu.edu/lennard-jones-potential/>
2. <https://levelup.gitconnected.com/graphics-pipeline-in-opengl-9db1c264f374/>